

Data Communication & Computer Networks

Course Instructor:
Rana Faisal Hayat

Application Layer: outline

1 principles of network applications

- app architectures
- app requirements

2 Web and HTTP

3 FTP

4 electronic mail

- SMTP, POP3, IMAP

5 DNS

6 socket process with UDP and TCP

Application layer

our goals:

- ❖ Implementation aspects of network application protocols
- ❖ learn about protocols by examining popular application-level protocols
 - HTTP
 - FTP
 - SMTP / POP3 / IMAP
 - DNS
- ❖ creating network applications
 - socket API

Some network apps

- ❖ e-mail
- ❖ web
- ❖ text messaging
- ❖ remote login
- ❖ P2P file sharing
- ❖ multi-user network games
- ❖ streaming stored video (YouTube, Hulu, Netflix)
- ❖ voice over IP (e.g., Skype)
- ❖ real-time video conferencing
- ❖ social networking
- ❖ search
- ❖ ...
- ❖ ...

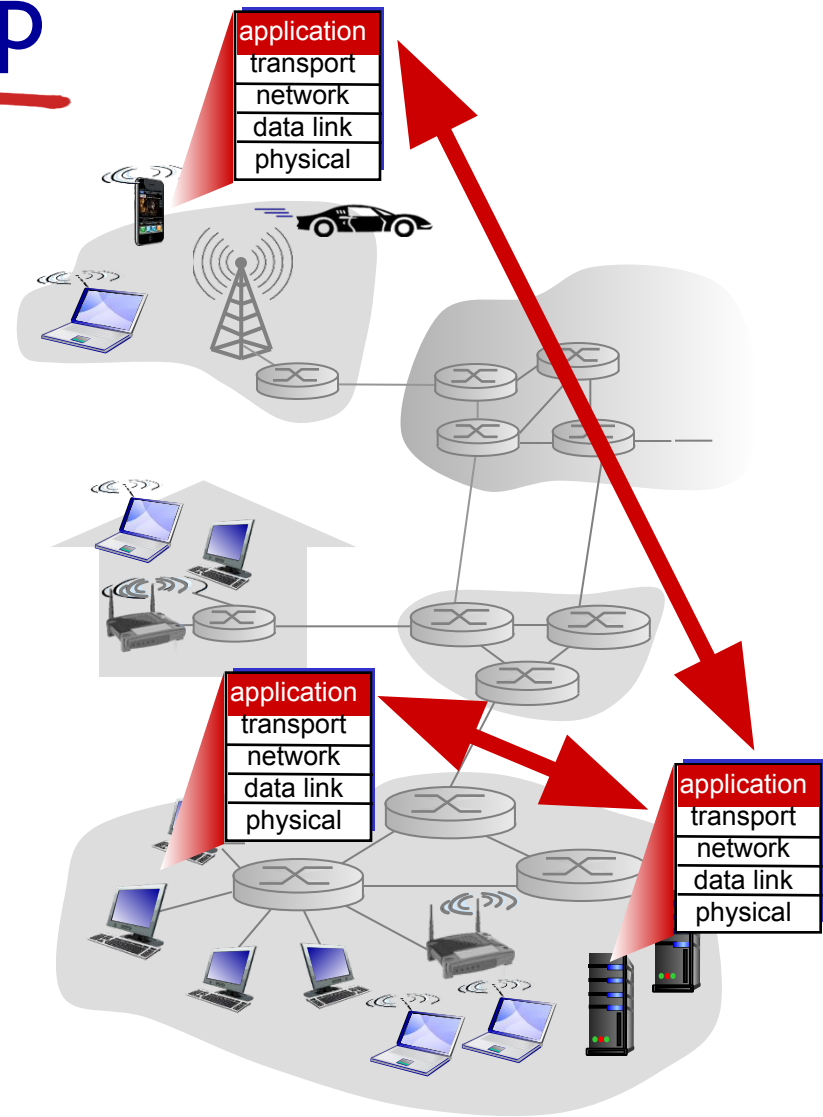
Creating a network app

write programs that:

- ❖ run on (different) *end systems*
- ❖ communicate over network
- ❖ e.g., web server software communicates with browser software

no need to write software for network-core devices

- ❖ network-core devices do not run user applications
- ❖ applications on end systems allows for rapid app development, propagation



Application architectures

possible structure of applications:

- ❖ client-server
- ❖ peer-to-peer (P2P)

Processes communicating

process: program running within a host

- ❖ within same host, two processes communicate using **inter-process communication** (defined by OS)
- ❖ processes in different hosts communicate by exchanging **messages**

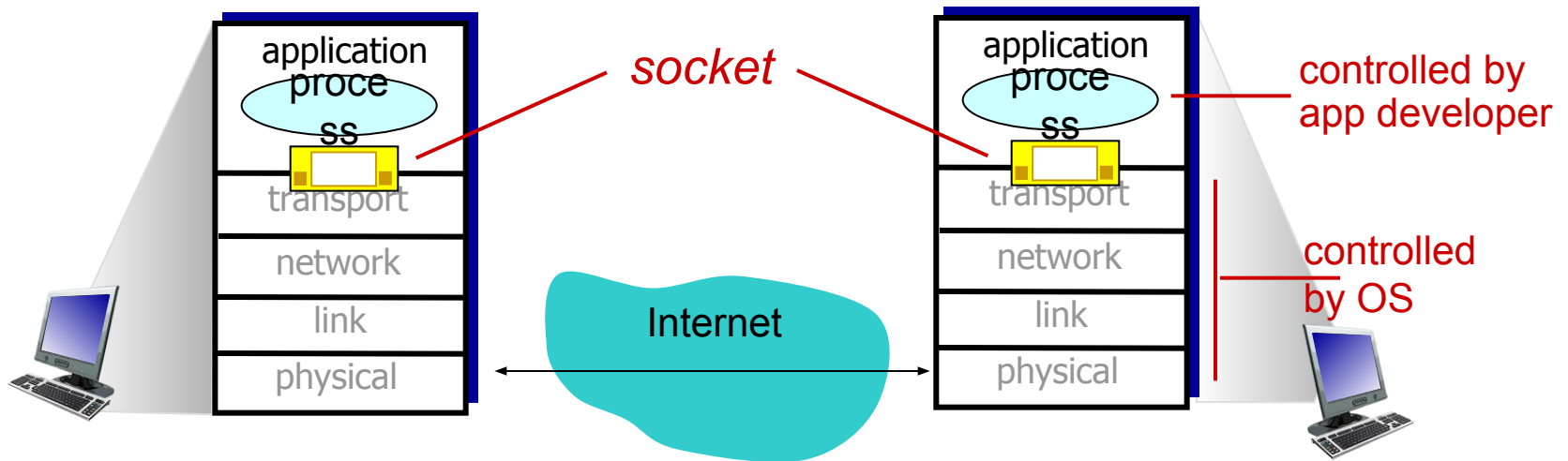
clients, servers

client process: process that initiates communication

server process: process that waits to be contacted

Sockets

- ❖ process sends/receives messages to/from its **socket**
- ❖ socket analogous to door
 - sending process pushes message out door
 - sending process **relies on transport infrastructure** on other side of door to deliver message to socket at receiving process



Addressing processes

- ❖ to receive messages, process must have *identifier*
- ❖ host device has unique 32-bit IP address
- ❖ Q: does IP address of host on which process runs serve for identifying the process?
 - A: no, *many* processes can be running on same host
- ❖ *identifier* includes both **IP address** and **port numbers** associated with process on host.
- ❖ example port numbers:
 - HTTP server: 80
 - mail server: 25
- ❖ to send HTTP message to `gaia.cs.umass.edu` web server:
 - **IP address**: 128.119.245.12
 - **port number**: 80
- ❖ more shortly...

App-layer protocol defines

- ❖ types of messages exchanged,
 - e.g., request, response
- ❖ message syntax:
 - what fields in messages & how fields are defined
- ❖ message semantics
 - meaning of information in fields
- ❖ rules for when and how processes send & respond to messages

What transport service does an app need?

data integrity

- ❖ some apps (e.g., file transfer, web transactions) require 100% reliable data transfer
- ❖ other apps (e.g., audio) can tolerate some loss

timing

- ❖ some apps (e.g., Internet telephony, interactive games) require low delay to be “effective”

throughput

- ❖ some apps (e.g., multimedia) require minimum amount of throughput to be “effective”

security

- ❖ encryption, data integrity,
...

Internet transport protocols services

TCP service:

- ❖ *reliable transport* between sending and receiving process
- ❖ *flow control*: sender won't overwhelm receiver
- ❖ *congestion control*: regulate sender when network overloaded
- ❖ *does not provide*: timing, minimum throughput guarantee, security
- ❖ *connection-oriented*: setup required between client and server processes

UDP service:

- ❖ *unreliable data transfer* between sending and receiving process
- ❖ *does not provide*: reliability, flow control, congestion control, timing, throughput guarantee, security, or connection setup,

Q: why bother? Why is there a UDP?

Internet apps: application, transport protocols

	application	application layer protocol	underlying transport protocol
	e-mail	SMTP [RFC 2821]	TCP
remote terminal access		Telnet [RFC 854]	TCP
	Web	HTTP [RFC 2616]	TCP
	file transfer	FTP [RFC 959]	TCP
streaming multimedia		HTTP (e.g., YouTube), RTP [RFC 1889]	TCP or UDP
Internet telephony		SIP, RTP, proprietary (e.g., Skype)	TCP or UDP

Securing TCP

TCP & UDP

- ❖ no encryption
- ❖ cleartext passwds sent into socket traverse Internet in cleartext

SSL(Secure Socket Layer)

- ❖ provides encrypted TCP connection
- ❖ data integrity
- ❖ end-point authentication

SSL is at app layer

- ❖ Apps use SSL libraries, which “talk” to TCP

SSL socket API

- ❖ cleartext passwds sent into socket traverse Internet encrypted

Application Layer: outline

1 principles of network applications

- app architectures
- app requirements

2 Web and HTTP

3 FTP

4 electronic mail

- SMTP, POP3, IMAP

5 DNS

6 socket programming with UDP and TCP

Web and HTTP

First, a review...

- ❖ *web page* consists of *objects*
- ❖ object can be HTML file, JPEG image, Java applet, audio file,...
- ❖ web page consists of *base HTML-file* which includes *several referenced objects*
- ❖ each object is addressable by a *URL*, e.g.,

`www.someschool.edu/someDept/pic.gif`

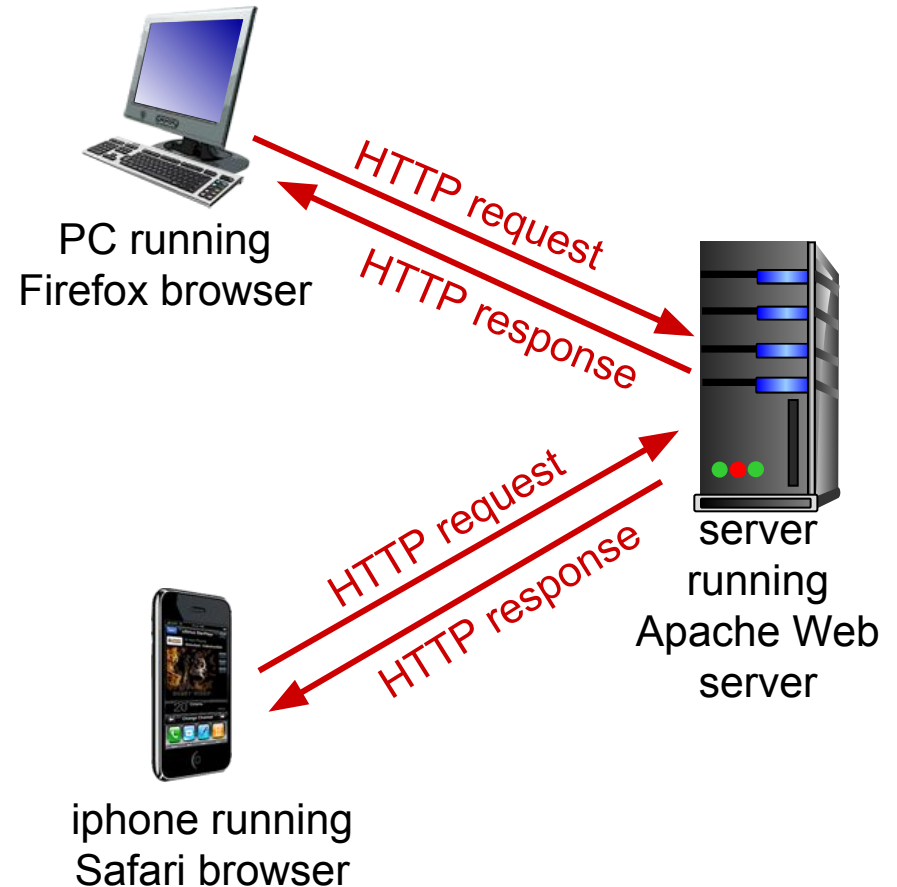
host name

path name

HTTP overview

HTTP: hypertext transfer protocol

- ❖ Web's application layer protocol
- ❖ client/server model
 - **client**: browser that requests, receives, (using HTTP protocol) and “displays” Web objects
 - **server**: Web server sends (using HTTP protocol) objects in response to requests



HTTP overview (continued)

uses TCP:

- ❖ client initiates TCP connection (creates socket) to server, port 80
- ❖ server accepts TCP connection from client
- ❖ HTTP messages (application-layer protocol messages) exchanged between browser (HTTP client) and Web server (HTTP server)
- ❖ TCP connection closed

HTTP is “stateless”

- ❖ server maintains no information about past client requests

HTTP connections

non-persistent HTTP

- ❖ at most one object sent over TCP connection
 - connection then closed

persistent HTTP

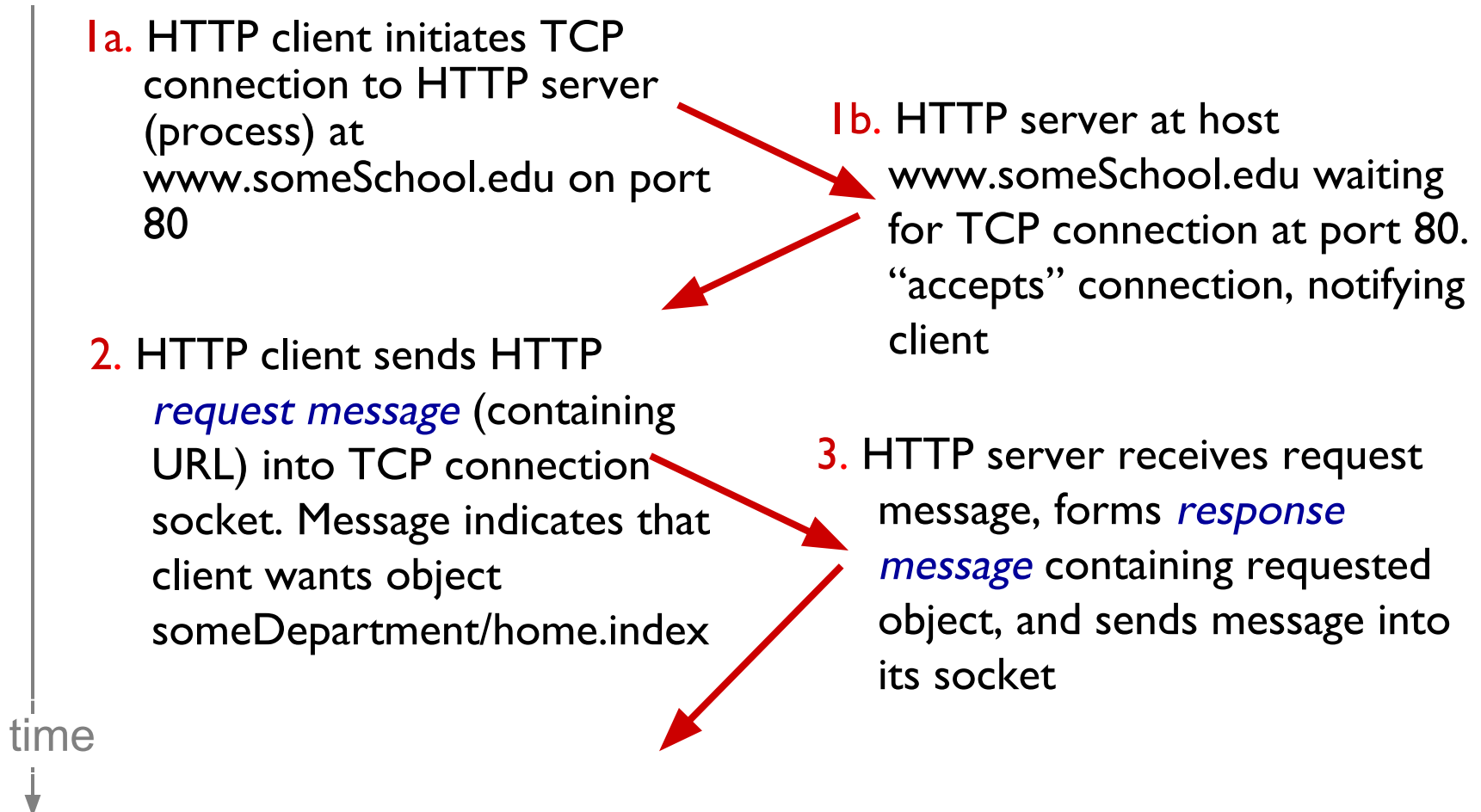
- ❖ multiple objects can be sent over single TCP connection between client, server

Non-persistent HTTP

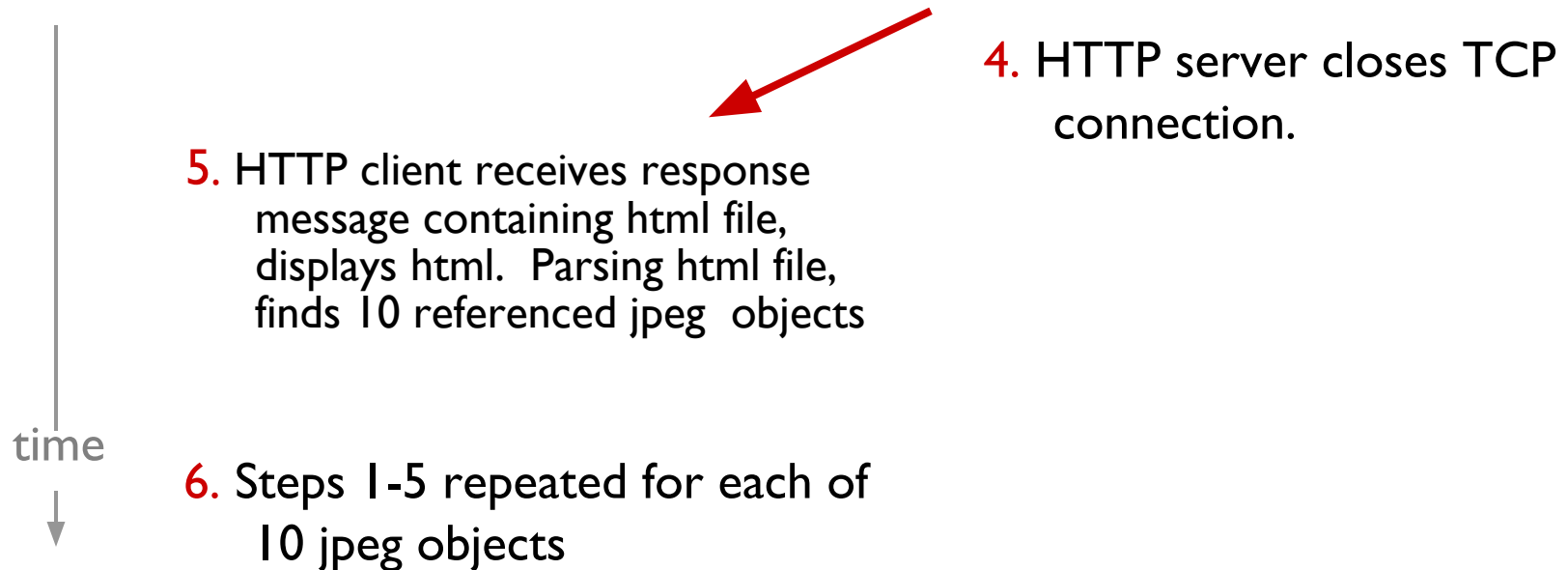
suppose user enters URL:

`www.someSchool.edu/someDepartment/home.index`

(contains text,
references to 10
jpeg images)



Non-persistent HTTP (cont.)



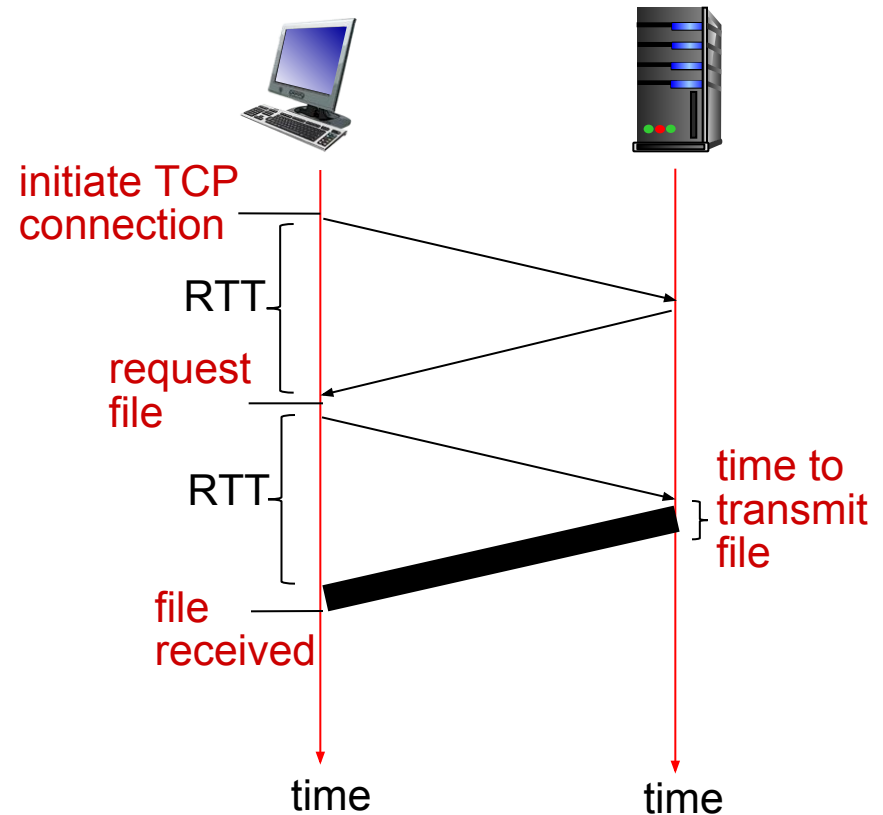
Non-persistent HTTP: response time

RTT(Round Trip Time)

(definition): time for a small packet to travel from client to server and back

HTTP response time:

- ❖ one RTT to initiate TCP connection
- ❖ one RTT for HTTP request and first few bytes of HTTP response to return
- ❖ file transmission time
- ❖ non-persistent HTTP response time =
 $2\text{RTT} + \text{file transmission time}$



Persistent HTTP

non-persistent HTTP issues:

- ❖ requires 2 RTTs per object
- ❖ OS overhead for *each* TCP connection
- ❖ browsers often open parallel TCP connections to fetch referenced objects

persistent HTTP:

- ❖ server leaves connection open after sending response
- ❖ subsequent HTTP messages between same client/server sent over open connection
- ❖ client sends requests as soon as it encounters a referenced object
- ❖ as little as one RTT for all the referenced objects

Q/A